

Program and output to implement producer and consumer problem using semaphores.

```
#include <stdio.h>
#include <stdlib.h>    // initialize a mutex to 1
int mutex = 1;        // buffer size and counters
int full = 0;
int empty;
int x = 0;             // produced item count
void producer(){      // function to produce an item and add to buffer
    --mutex;           // lock mutex
    ++full;            // increase full slots
    --empty;           // decrease empty slots
    x++;              // produce item
    printf("\nproducer produces item %d", x);
    ++mutex;           // unlock mutex
} // function to consume an item and remove from buffer
void consumer(){
    --mutex;           // lock mutex
    --full;            // decrease full slots
    ++empty;           // increase empty slots
    printf("\nconsumer consumes item %d", x);
    x--;              // consume item
    ++mutex;           // unlock mutex
} // driver code
int main(){
    int n, i;
    printf("Enter buffer size: ");
    scanf("%d", &empty); // initialize empty slots as buffer size
    printf("\n1. press 1 for producer");
    printf("\n2. press 2 for consumer");
    printf("\n3. press 3 for exit");
    for (i = 1; i > 0; i++) {
        printf("\n\nEnter your choice: ");
        scanf("%d", &n);
        switch (n) {
            case 1:
                if (mutex == 1 && empty != 0) {
                    producer();
                }
            else {
                printf("buffer is full!");
            }
        }
        break;
    }
}
```

```

case 2:
    if (mutex == 1 && full != 0) {
        consumer();
    }
    else {
        printf("buffer is empty!");
    }
    break;
case 3:
    printf("Exiting program.\n");
    exit(0);
    break;
default:
    printf("invalid choice!");
    break;
}
}
return 0;
}

```

```

C:\Users\pbasa\OneDrive\De...
Enter buffer size: 2

1. press 1 for producer
2. press 2 for consumer
3. press 3 for exit
Enter your choice: 1

producer produces item 1
Enter your choice: 1

producer produces item 2
Enter your choice: 1
buffer is full!
Enter your choice: 2

consumer consumes item 2
Enter your choice: 1

producer produces item 2
Enter your choice: 1
buffer is full!
Enter your choice: 3
Exiting program.

-----
Process exited after 31.56 seconds with return value 0
Press any key to continue . . .

```

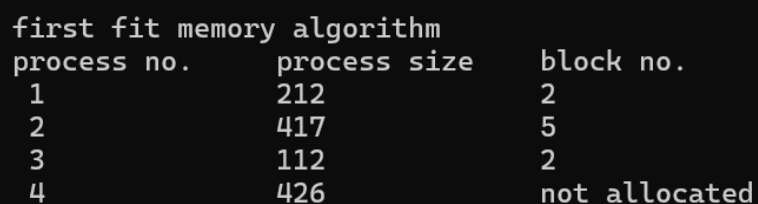
Lab 3: Implementation of Memory Fit algorithm in C (First, worst and best)

First Fit

```
#include <stdio.h> // first-fit memory allocation

void firstfit(int blocksize[], int m, int processsize[], int n) {
    int allocation[n]; // initialize allocations to -1 (not allocated)
    for(int i = 0; i < n; i++)
        allocation[i] = -1;
    // allocate each process to the first suitable block
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {
            if(blocksize[j] >= processsize[i]) {
                allocation[i] = j; // assign block
                blocksize[j] -= processsize[i]; // reduce block size
                break; // next process
            }
        }
    }
    printf("first fit memory algorithm\n");
    printf("process no.\tprocess size\tblock no.\n"); // print process allocation
    for(int i = 0; i < n; i++) {
        printf(" %d\t\t%d\t\t", i+1, processsize[i]);
        if(allocation[i] != -1)
            printf("%d\n", allocation[i]+1); // allocated
        else
            printf("not allocated\n"); // no block found
    }
}

int main() {
    int blocksize[] = {100, 500, 200, 300, 600};
    int processsize[] = {212, 417, 112, 426};
    int m = sizeof(blocksize)/sizeof(blocksize[0]);
    int n = sizeof(processsize)/sizeof(processsize[0]);
    firstfit(blocksize, m, processsize, n);
    return 0;
}
```



process no.	process size	block no.
1	212	2
2	417	5
3	112	2
4	426	not allocated

```
-----
Process exited after 0.06723 seconds with return value 0
Press any key to continue . . . |
```

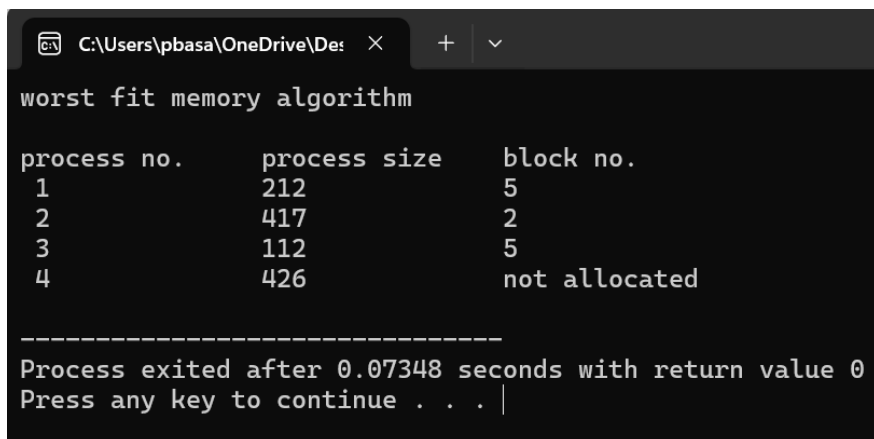
Worst Fit

```
#include <stdio.h>
#include <string.h> // worst-fit memory allocation
void worstfit(int blocksize[], int m, int processsize[], int n) {
    int allocation[n];
    memset(allocation, -1, sizeof(allocation)); // no block assigned initially
                                                // allocate each process to the largest available block

    for(int i = 0; i < n; i++) {
        int wstIdx = -1;
        for(int j = 0; j < m; j++) {
            if(blocksize[j] >= processsize[i]) {
                if(wstIdx == -1 || blocksize[j] > blocksize[wstIdx])
                    wstIdx = j; // find largest suitable block
            }
        }
        if(wstIdx != -1) {
            allocation[i] = wstIdx;
            blocksize[wstIdx] -= processsize[i]; // reduce block size
        }
    }

    printf("first fit memory algorithm\n");
    printf("\nprocess no.\tprocess size\tblock no.\n"); // print allocation
    for(int i = 0; i < n; i++) {
        printf(" %d\t\t%d\t\t", i+1, processsize[i]);
        if(allocation[i] != -1)
            printf("%d\n", allocation[i]+1);
        else
            printf("not allocated\n");
    }
}

int main() {
    int blocksize[] = {100, 500, 200, 300, 600};
    int processsize[] = {212, 417, 112, 426};
    int m = sizeof(blocksize)/sizeof(blocksize[0]);
    int n = sizeof(processsize)/sizeof(processsize[0]);
    worstfit(blocksize, m, processsize, n);
    return 0;
}
```



```
worst fit memory algorithm

process no.      process size      block no.
1                212              5
2                417              2
3                112              5
4                426              not allocated

-----
Process exited after 0.07348 seconds with return value 0
Press any key to continue . . .
```

Best Fit

```
#include <stdio.h>
#include <string.h>

// best-fit memory

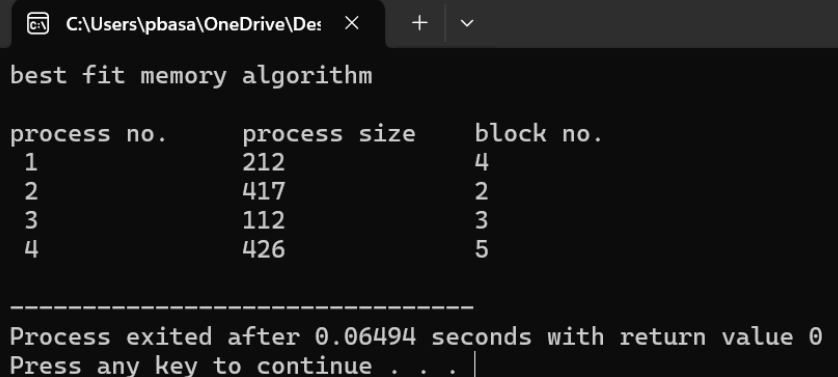
allocation
void bestfit(int blocksize[], int m, int processsize[], int n) {
    int allocation[n];
    memset(allocation, -1, sizeof(allocation)); // no block assigned initially
    // allocate each process to the smallest suitable block

    block
    for(int i = 0; i < n; i++) {
        int bestIdx = -1;
        for(int j = 0; j < m; j++) {
            if(blocksize[j] >= processsize[i]) {
                if(bestIdx == -1 || blocksize[j] < blocksize[bestIdx])
                    bestIdx = j; // find smallest suitable block
            }
        }
        if(bestIdx != -1) {
            allocation[i] = bestIdx;
            blocksize[bestIdx] -= processsize[i]; // reduce block size
        }
    }
    printf("best fit memory algorithm\n");

    // print
    process allocation

    printf("\nprocess no.\tprocess size\tblock no.\n");
    for(int i = 0; i < n; i++) {
        printf(" %d\t\t%d\t\t", i+1, processsize[i]);
        if(allocation[i] != -1)
            printf("%d\n", allocation[i]+1);
        else
            printf("not allocated\n");
    }
}

int main() {
    int blocksize[] = {100, 500, 200, 300, 600};
    int processsize[] = {212, 417, 112, 426};
    int m = sizeof(blocksize)/sizeof(blocksize[0]);
    int n = sizeof(processsize)/sizeof(processsize[0]);
    bestfit(blocksize, m, processsize, n);
    return 0;
}
```



```
best fit memory algorithm

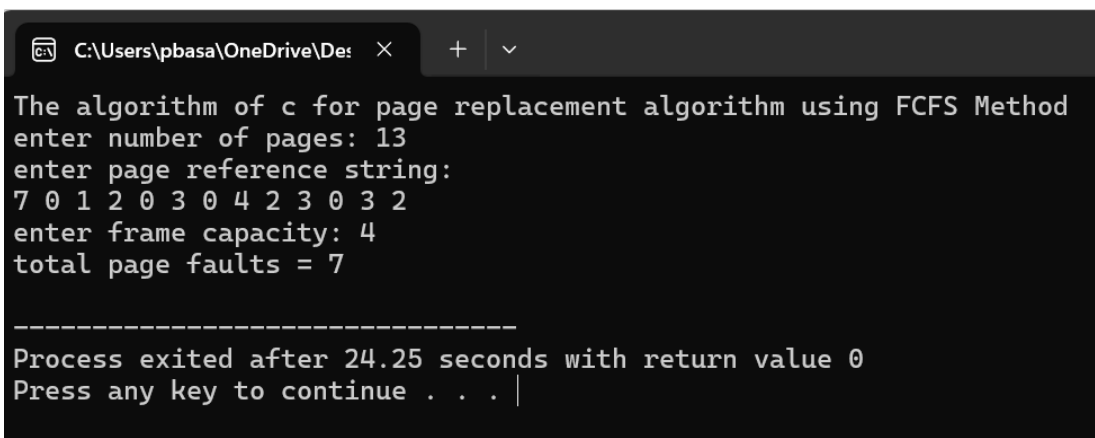
process no.      process size      block no.
1                212                4
2                417                2
3                112                3
4                426                5

-----
Process exited after 0.06494 seconds with return value 0
Press any key to continue . . . |
```

Lab 4: Implementation of page replacement algorithm in C (FCFS, LRU, Optimal)

FCFS(First Come First Serve)

```
#include <stdio.h>
int main() {
    int n, capacity; // input number of pages
    printf("enter number of pages: ");
    scanf("%d", &n);
    int pages[n]; // input page reference string
    printf("enter page reference string:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    } // input frame capacity
    printf("enter frame capacity: ");
    scanf("%d", &capacity);
    int frames[capacity];
    int pagefaults = 0;
    int front = 0; // initialize frames with -1 (empty)
    for (int i = 0; i < capacity; i++) {
        frames[i] = -1;
    }
    for (int i = 0; i < n; i++) {
        int found = 0; // check if page already exists
        for (int j = 0; j < capacity; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        } // if page not found → page fault
        if (!found) {
            frames[front] = pages[i];
            front = (front + 1) % capacity; // fifo rotation
            pagefaults++;
        }
    }
    printf("total page faults = %d\n", pagefaults);
    return 0;
}
```



The screenshot shows a Windows command prompt window with the following text:

```
C:\Users\pbasa\OneDrive\Desktop> gcc lab4.c -o lab4.exe
C:\Users\pbasa\OneDrive\Desktop> lab4.exe
The algorithm of c for page replacement algorithm using FCFS Method
enter number of pages: 13
enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2
enter frame capacity: 4
total page faults = 7

-----
Process exited after 24.25 seconds with return value 0
Press any key to continue . . . |
```

LRU

```
#include <stdio.h>
#include <limits.h>
int pagefaults(int pages[], int n, int capacity){
    int frames[capacity];
    int indexes[capacity];
    int page_faults = 0;
    for (int i = 0; i < capacity; i++) {
        frames[i] = -1;
        indexes[i] = -1;
    }
    for (int i = 0; i < n; i++)
    {
        int found = 0;
        for (int j = 0; j < capacity; j++){
            if (frames[j] == pages[i]) {
                found = 1;
                indexes[j] = i;
                break;
            }
        }
        // if page not found, page fault occurs
        if (!found)
        {
            int lru = INT_MAX;
            int pos = -1;
            for (int j = 0; j < capacity; j++){
                if (frames[j] == -1) {
                    pos = j;
                    break;
                }
                if (indexes[j] < lru) {
                    lru = indexes[j];
                    pos = j;
                }
            }
            frames[pos] = pages[i];
            indexes[pos] = i;
            page_faults++;
        }
    }
    return page_faults;
}
```

// function to find page faults using lru

// to store current pages

// to store recent use index

// initialize frames as empty

// check if page is already present

// update recent use

// find least recently used page

// replace lru page

// driver code

```

int main(){
    int n, capacity;
    printf("The algorithm of c for page replacement algorithm using LRU Method\n");
    printf("enter number of pages: ");
    scanf("%d", &n);
    int pages[n];
    printf("enter page reference string:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }
    printf("enter frame capacity: ");
    scanf("%d", &capacity);
    printf("total page faults = %d\n", pagefaults(pages, n, capacity));
    return 0;
}

```

```

C:\Users\pbasa\OneDrive\Desktop
The algorithm of c for page replacement algorithm using LRU Method
enter number of pages: 13
enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2
enter frame capacity: 4
total page faults = 6

-----
Process exited after 10.95 seconds with return value 0
Press any key to continue . . .

```


Optimal

```
// c program to demonstrate optimal page
// replacement algorithm with user input

#include <stdio.h>
#include <limits.h>

int search(int key, int fr[], int fsize){
    for (int i = 0; i < fsize; i++) {
        if (fr[i] == key)
            return 1;
    }
    return 0;
}

// function to check whether a page exists in frame

// function to find the page that will not be used
// recently in future

int predict(int pg[], int fr[], int pn, int index, int fsize){
    int res = -1, farthest = index;
    for (int i = 0; i < fsize; i++) {
        int j;
        for (j = index; j < pn; j++) {
            if (fr[i] == pg[j]) {
                if (j > farthest) {
                    farthest = j;
                    res = i;
                }
            }
            break;
        }
    }
    // if page is never used again
    if (j == pn)
        return i;
    return (res == -1) ? 0 : res;
}

// function to implement optimal page replacement

void optimalpage(int pg[], int pn, int fn){
    int fr[fn];
    int fsize = 0;
    int hit = 0;
    // initialize frames as empty
    for (int i = 0; i < fn; i++) {
        fr[i] = -1;
    }
    // page hit
    for (int i = 0; i < pn; i++) {
        if (search(pg[i], fr, fsize)) {
            hit++;
            continue;
        }
    }
}
```

```

    }
    if (frsize < fn) {
        fr[frsize++] = pg[i];
    }
    else {
        int j = predict(pg, fr, pn, i + 1, fn);
        fr[j] = pg[i];
    }
}
printf("no. of hits = %d\n", hit);
printf("no. of misses = %d\n", pn - hit);
}
// driver code
int main() {
    int pn, fn;
    printf("enter number of pages: ");
    scanf("%d", &pn);
    int pg[pn];
    printf("enter page reference string:\n");
    for (int i = 0; i < pn; i++) {
        scanf("%d", &pg[i]);
    }
    printf("enter number of frames: ");
    scanf("%d", &fn);
    optimalpage(pg, pn, fn);
    return 0;
}

```

```

C:\Users\pbasa\OneDrive\Desktop >
enter number of pages: 13
enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2
enter number of frames: 4
no. of hits = 7
no. of misses = 6

-----
Process exited after 40.86 seconds with return value 0
Press any key to continue . . .

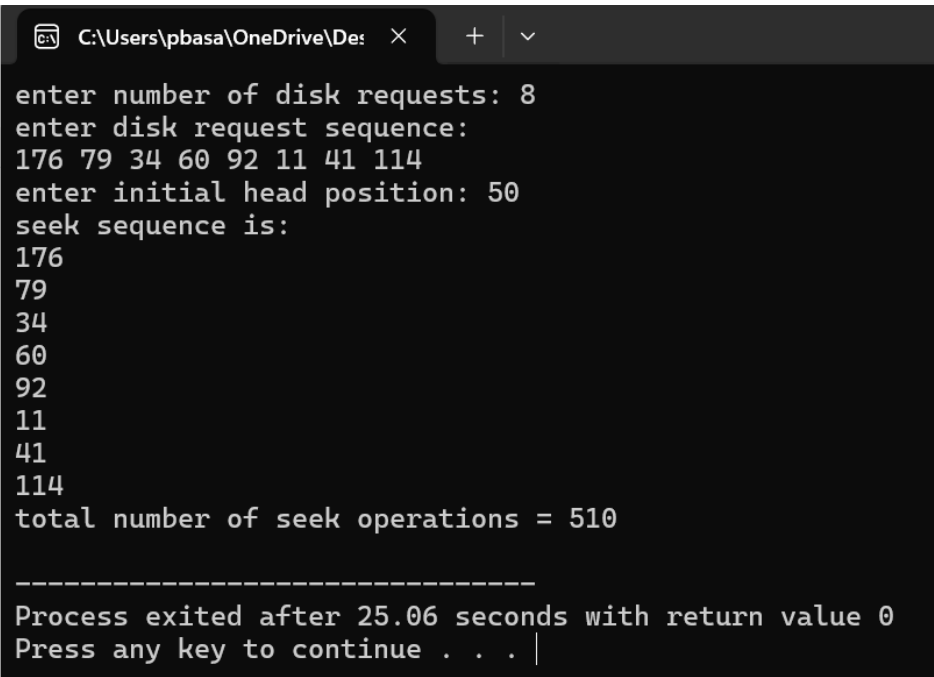
```

Lab -5: Implementation of disk scheduling algorithm in C

FCFS(Disk Scheduling Algorithm)

```
#include <stdio.h>
#include <stdlib.h>
void fcfs(int arr[], int size, int head){
    int seek_count = 0;
    int cur_track, distance;
    printf("seek sequence is:\n");
    for (int i = 0; i < size; i++){
        cur_track = arr[i];                // calculate absolute distance
        distance = abs(head - cur_track);  // increase the total seek count
        seek_count += distance;            // accessed track becomes new head
        head = cur_track;
        printf("%d\n", cur_track);
    }
    printf("total number of seek operations = %d\n", seek_count);
}                                           // driver code

int main(){
    int size, head;
    printf("enter number of disk requests: ");
    scanf("%d", &size);
    int arr[size];
    printf("enter disk request sequence:\n");
    for (int i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }
    printf("enter initial head position: ");
    scanf("%d", &head);
    fcfs(arr, size, head);
    return 0;
}
```



```
C:\Users\pbasa\OneDrive\Des  ×  +  ▾

enter number of disk requests: 8
enter disk request sequence:
176 79 34 60 92 11 41 114
enter initial head position: 50
seek sequence is:
176
79
34
60
92
11
41
114
total number of seek operations = 510

-----
Process exited after 25.06 seconds with return value 0
Press any key to continue . . . |
```

SSTF(Disk Scheduling Algorithm)

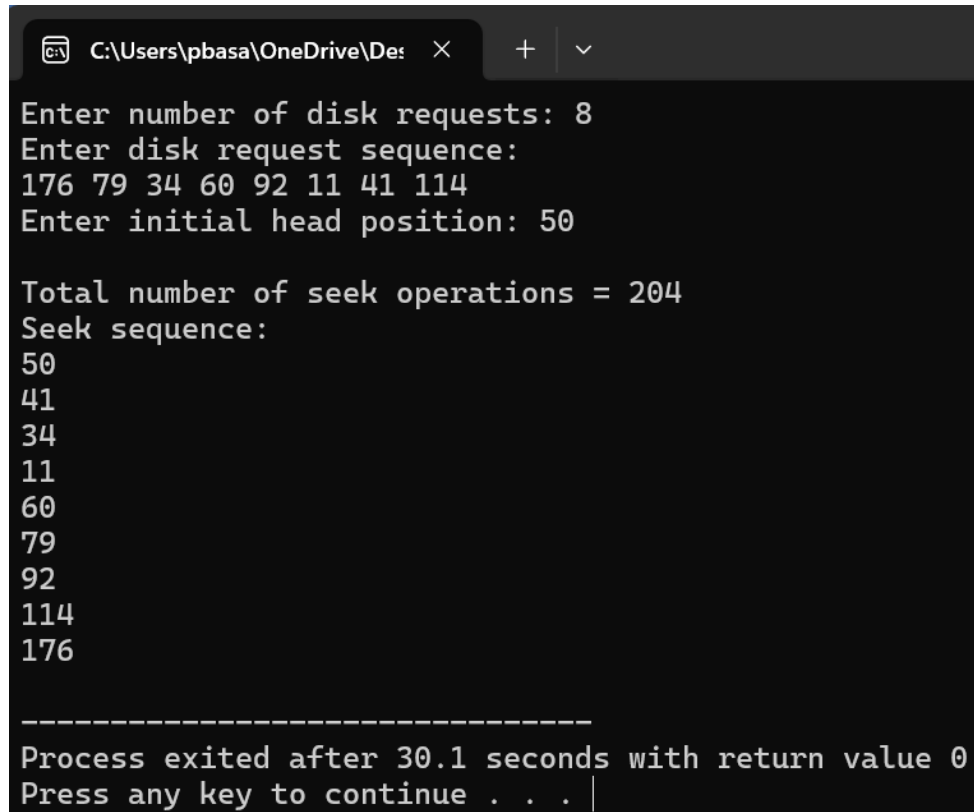
```
#include <stdio.h>
#include <stdlib.h>
void calculateDifference(int request[], int head, int diff[][2], int n){
    for (int i = 0; i < n; i++){
        diff[i][0] = abs(head - request[i]); // distance
    }
}
int findMin(int diff[][2], int n){
    int min = 100000;
    int index = -1;
    for (int i = 0; i < n; i++){
        {
            if (diff[i][1] == 0 && diff[i][0] < min){
                min = diff[i][0];
                index = i;
            }
        }
    }
    return index;
}
void SSTF(int request[], int n, int head){
    int diff[n][2];
    int seekSequence[n + 1];
    int seekCount = 0;
    // initialize diff array
    for (int i = 0; i < n; i++){
        diff[i][0] = 0; // distance
        diff[i][1] = 0; // accessed flag
    }
    seekSequence[0] = head;
    for (int i = 0; i < n; i++){
        calculateDifference(request, head, diff, n);
        int index = findMin(diff, n);

        diff[index][1] = 1; // mark as accessed
        seekCount += diff[index][0]; // add seek time
        head = request[index]; // update head
        seekSequence[i + 1] = head;
    }
    printf("\nTotal number of seek operations = %d\n", seekCount);
    printf("Seek sequence:\n");
```

```

    for (int i = 0; i <= n; i++){
        printf("%d\n", seekSequence[i]);
    }
}
int main(){
    int n, head;
    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    int request[n];
    printf("Enter disk request sequence:\n");
    for (int i = 0; i < n; i++){
        scanf("%d", &request[i]);
    }
    printf("Enter initial head position: ");
    scanf("%d", &head);
    SSTF(request, n, head);
    return 0;
}

```



```

C:\Users\pbasa\OneDrive\De:
Enter number of disk requests: 8
Enter disk request sequence:
176 79 34 60 92 11 41 114
Enter initial head position: 50

Total number of seek operations = 204
Seek sequence:
50
41
34
11
60
79
92
114
176

-----
Process exited after 30.1 seconds with return value 0
Press any key to continue . . .

```

SCAN(Disk Scheduling Algorithm)

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int size, disk_size = 200;
    printf("Enter number of disk requests: ");
    scanf("%d", &size);
    int arr[size];
    printf("Enter the disk requests: ");
    for(int i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }
    int head;
    printf("Enter initial head position: ");
    scanf("%d", &head);
    char direction[10];
    printf("Enter initial direction (left/right): ");
    scanf("%s", direction);
    int seek_count = 0, distance, cur_track;
    int left[size + 1], right[size + 1];
    int left_count = 0, right_count = 0;
    int seek_sequence[size + 2];
    int seq_index = 0;
    if (strcmp(direction, "left") == 0) {
        left[left_count++] = 0;
    } else if (strcmp(direction, "right") == 0) {
        right[right_count++] = disk_size - 1;
    }
    // Separate requests into left and right
    for(int i = 0; i < size; i++) {
        if(arr[i] < head)
            left[left_count++] = arr[i];
        else if(arr[i] > head)
            right[right_count++] = arr[i];
    }
    // Sort left in ascending order
    for(int i = 0; i < left_count-1; i++) {
        for(int j = i+1; j < left_count; j++) {
            if(left[i] > left[j]) {
                int temp = left[i];
                left[i] = left[j];
                left[j] = temp;
            }
        }
    }
}
```

```

} // Sort right in ascending order
for(int i = 0; i < right_count-1; i++) {
    for(int j = i+1; j < right_count; j++) {
        if(right[i] > right[j]) {
            int temp = right[i];
            right[i] = right[j];
            right[j] = temp;
        }
    }
}

// Run SCAN algorithm
int run = 2;
while(run--) {
    if(strcmp(direction, "left") == 0) {
        for(int i = left_count - 1; i >= 0; i--) {
            cur_track = left[i];
            seek_sequence[seq_index++] = cur_track;
            distance = abs(cur_track - head);
            seek_count += distance;
            head = cur_track;
        }
        strcpy(direction, "right");
    } else if(strcmp(direction, "right") == 0) {
        for(int i = 0; i < right_count; i++) {
            cur_track = right[i];
            seek_sequence[seq_index++] = cur_track;
            distance = abs(cur_track - head);
            seek_count += distance;
            head = cur_track;
        }
        strcpy(direction, "left");
    }
}

printf("Total number of seek operations = %d\n", seek_count);
printf("Seek Sequence is: ");
for(int i = 0; i < seq_index; i++) {
    printf("%d ", seek_sequence[i]);
}
printf("\n");
return 0;
}

```

```
C:\Users\pbasa\OneDrive\De: X + v
SCAN disk scheduling algorithm
Enter total disk size: 200
Enter number of disk requests: 8
Enter disk request sequence:
176 79 34 60 92 11 41 114
Enter initial head position: 50
Enter initial direction (left/right): right

Total number of seek operations = 337
Seek sequence:
50
60
79
92
114
176
199
41
34
11

-----
Process exited after 24.02 seconds with return value 0
Press any key to continue . . . |
```

```
C:\Users\pbasa\OneDrive\De: X + v
SCAN disk scheduling algorithm
Enter total disk size: 200
Enter number of disk requests: 8
Enter disk request sequence:
176 79 34 60 92 11 41 114
Enter initial head position: 50
Enter initial direction (left/right): left

Total number of seek operations = 226
Seek sequence:
50
41
34
11
0
60
79
92
114
176

-----
Process exited after 34.37 seconds with return value 0
Press any key to continue . . . |
```


C-SCAN(Disk Scheduling Algorithm)

```
#include <stdio.h>
#include <stdlib.h>
void CSCAN(int request[], int n, int head, int disk_size) {
    int seekSequence[n + 2];           // sequence of visited tracks
    int seekCount = 0;
    int left[n + 1], right[n + 1];
    int left_count = 0, right_count = 0;
    int seq_index = 0;                 // Append end values
    left[left_count++] = 0;            // left end
    right[right_count++] = disk_size - 1; // right end

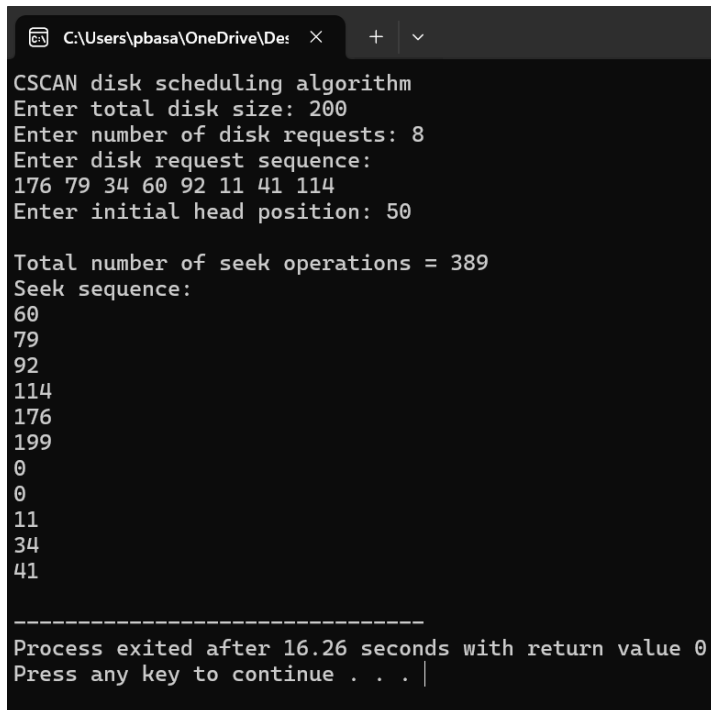
    for (int i = 0; i < n; i++) {      // Separate requests into left and right of head
        if (request[i] < head)
            left[left_count++] = request[i];
        else if (request[i] > head)
            right[right_count++] = request[i];
    } // Sort left in ascending order
    for (int i = 0; i < left_count - 1; i++)
        for (int j = i + 1; j < left_count; j++)
            if (left[i] > left[j]) {
                int temp = left[i];
                left[i] = left[j];
                left[j] = temp;
            } // Sort right in ascending order
    for (int i = 0; i < right_count - 1; i++)
        for (int j = i + 1; j < right_count; j++)
            if (right[i] > right[j]) {
                int temp = right[i];
                right[i] = right[j];
                right[j] = temp;
            } // Service requests on the right of head first
    for (int i = 0; i < right_count; i++) {
        seekCount += abs(head - right[i]);
        head = right[i];
        seekSequence[seq_index++] = head;
    } // Jump to the beginning (0)
    seekCount += abs(head - 0);
    head = 0;
    seekSequence[seq_index++] = head; // Now service requests on the left
    for (int i = 0; i < left_count; i++) {
        seekCount += abs(head - left[i]);
```

```

        head = left[i];
        seekSequence[seq_index++] = head;
    }
    // Print results
    printf("\nTotal number of seek operations = %d\n", seekCount);
    printf("Seek sequence:\n");
    for (int i = 0; i < seq_index; i++)
        printf("%d\n", seekSequence[i]);
}

int main() {
    int n, head, disk_size;
    printf("CSCAN disk scheduling algorithm\n");
    printf("Enter total disk size: ");
    scanf("%d", &disk_size);
    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    int request[n];
    printf("Enter disk request sequence:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &request[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    CSCAN(request, n, head, disk_size);
    return 0;
}

```



```

C:\Users\pbasa\OneDrive\Desktop
CSCAN disk scheduling algorithm
Enter total disk size: 200
Enter number of disk requests: 8
Enter disk request sequence:
176 79 34 60 92 11 41 114
Enter initial head position: 50

Total number of seek operations = 389
Seek sequence:
60
79
92
114
176
199
0
0
11
34
41

-----
Process exited after 16.26 seconds with return value 0
Press any key to continue . . . |

```

LOOK(Disk Scheduling Algorithm)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
void LOOK(int request[], int n, int head, char direction[]) {
    int seekSequence[n];                // sequence of visited tracks
    int seekCount = 0;
    int left[n], right[n];
    int left_count = 0, right_count = 0;
    int seq_index = 0;                  // Separate requests into left and right of head
    for (int i = 0; i < n; i++) {
        if (request[i] < head)
            left[left_count++] = request[i];
        else if (request[i] > head)
            right[right_count++] = request[i];
    }                                    // Sort left in ascending order
    for (int i = 0; i < left_count - 1; i++)
        for (int j = i + 1; j < left_count; j++)
            if (left[i] > left[j]) {
                int temp = left[i];
                left[i] = left[j];
                left[j] = temp;
            } // Sort right in ascending order
    for (int i = 0; i < right_count - 1; i++)
        for (int j = i + 1; j < right_count; j++)
            if (right[i] > right[j]) {
                int temp = right[i];
                right[i] = right[j];
                right[j] = temp;
            }
    int run = 2;                        // to scan in both directions
    while (run-- > 0) {
        if (strcmp(direction, "left") == 0) {
            for (int i = left_count - 1; i >= 0; i--) {
                seekCount += abs(head - left[i]);
                head = left[i];
                seekSequence[seq_index++] = head;
            }
            strcpy(direction, "right");
        } else if (strcmp(direction, "right") == 0) {
            for (int i = 0; i < right_count; i++) {
                seekCount += abs(head - right[i]);
            }
        }
    }
}
```

```

        head = right[i];
        seekSequence[seq_index++] = head;
    }
    strcpy(direction, "left");
}
}
printf("\nTotal number of seek operations = %d\n", seekCount);
printf("Seek sequence:\n");
for (int i = 0; i < seq_index; i++)
    printf("%d\n", seekSequence[i]);
}
int main() {
    int n, head;
    char direction[10];
    printf("LOOK disk scheduling algorithm\n ");
    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    int request[n];
    printf("Enter disk request sequence:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &request[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter initial direction (left/right): ");
    scanf("%s", direction);
    LOOK(request, n, head, direction);
    return 0;
}

```

```

C:\Users\pbasa\OneDrive\Des X + v
LOOK disk scheduling algorithm
Enter number of disk requests: 8
Enter disk request sequence:
176 79 34 60 92 11 41 114
Enter initial head position: 50
Enter initial direction (left/right): right

Total number of seek operations = 361
Seek sequence:
60
60
60
92
114
79
176
11

-----
Process exited after 18.71 seconds with return value 0
Press any key to continue . . .

```

```

C:\Users\pbasa\OneDrive\Des X + v
LOOK disk scheduling algorithm
Enter number of disk requests: 8
Enter disk request sequence:
176 79 34 60 92 11 41 114
Enter initial head position: 50
Enter initial direction (left/right): left

Total number of seek operations = 394
Seek sequence:
79
176
11
60
60
60
92
114

-----
Process exited after 24.5 seconds with return value 0
Press any key to continue . . .

```

C-LOOK(Disk Scheduling Algorithm)

```
#include <stdio.h>
#include <stdlib.h>
void CLOOK(int request[], int n, int head) {
    int seekSequence[n];                // sequence of visited tracks
    int seekCount = 0;
    int left[n], right[n];
    int left_count = 0, right_count = 0;
    int seq_index = 0;                // Separate requests into left and right of head
    for (int i = 0; i < n; i++) {
        if (request[i] < head)
            left[left_count++] = request[i];
        else if (request[i] > head)
            right[right_count++] = request[i];
    } // Sort left in ascending order
    for (int i = 0; i < left_count - 1; i++)
        for (int j = i + 1; j < left_count; j++)
            if (left[i] > left[j]) {
                int temp = left[i];
                left[i] = left[j];
                left[j] = temp;
            }
    // Sort right in ascending order
    for (int i = 0; i < right_count - 1; i++)
        for (int j = i + 1; j < right_count; j++)
            if (right[i] > right[j]) {
                int temp = right[i];
                right[i] = right[j];
                right[j] = temp;
            }
    // Service requests on the right first
    for (int i = 0; i < right_count; i++) {
        seekCount += abs(head - right[i]);
        head = right[i];
        seekSequence[seq_index++] = head;
    }
    if (left_count > 0) {
        // Jump to the first request on the left
        seekCount += abs(head - left[0]);
        head = left[0];
    }
    // Now service requests on the left
    for (int i = 0; i < left_count; i++) {
        seekCount += abs(head - left[i]);
        head = left[i];
        seekSequence[seq_index++] = head;
    }
}
```

```

    }
    // Print results
    printf("\nTotal number of seek operations = %d\n", seekCount);
    printf("Seek sequence:\n");
    for (int i = 0; i < seq_index; i++)
        printf("%d\n", seekSequence[i]);
}
int main() {
    int n, head;
    printf("C-LOOK disk scheduling algorithm \n");
    printf("Enter number of disk requests: ");
    scanf("%d", &n);
    int request[n];
    printf("Enter disk request sequence:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &request[i]);
    printf("Enter initial head position: ");
    scanf("%d", &head);
    CLOOK(request, n, head);
    return 0;
}

```

```

C:\Users\pbasa\OneDrive\Des > C-LOOK disk scheduling algorithm
Enter number of disk requests: 8
Enter disk request sequence:
176 79 34 60 92 11 41 114
Enter initial head position: 50

Total number of seek operations = 321
Seek sequence:
60
79
92
114
176
11
34
41

-----
Process exited after 26.46 seconds with return value 0
Press any key to continue . . . |

```